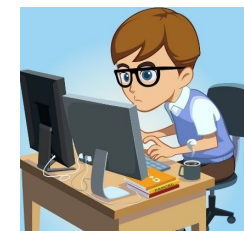




CodeBlocks调试功能快捷教程

1010101000010010101010101010101010100101000011001100110101010000100101010101010101010101001000011001100110101010000100101010101010101010101001010000110011001101010100001001010101010101010101010010100001100

制作 贺利坚





为什么要用单步调试

🖱️ 单步调试是发现运行错误和逻辑错误的“利器”，可用于

(1) 跟踪程序的执行流程，发现错误的线索

——发现该走A路径，却走了B路径

(2) 跟踪过程中，还可以观察变量的变化，从而发现其中存在的问题

——该是 1，执行中却分明是 2，之前哪儿出了问题？

🖱️ 单步执行除了可以帮助我们发现错误，对于初学者，还可以帮助我们理解语言的机制。

🖱️ “工欲善其事，必先利其器”，单步调试就是程序设计者最重要的工具之一

📁 这种**工具**的形态是软件。**程序员用软件当工具**，正常得不得了。用好这种工具！

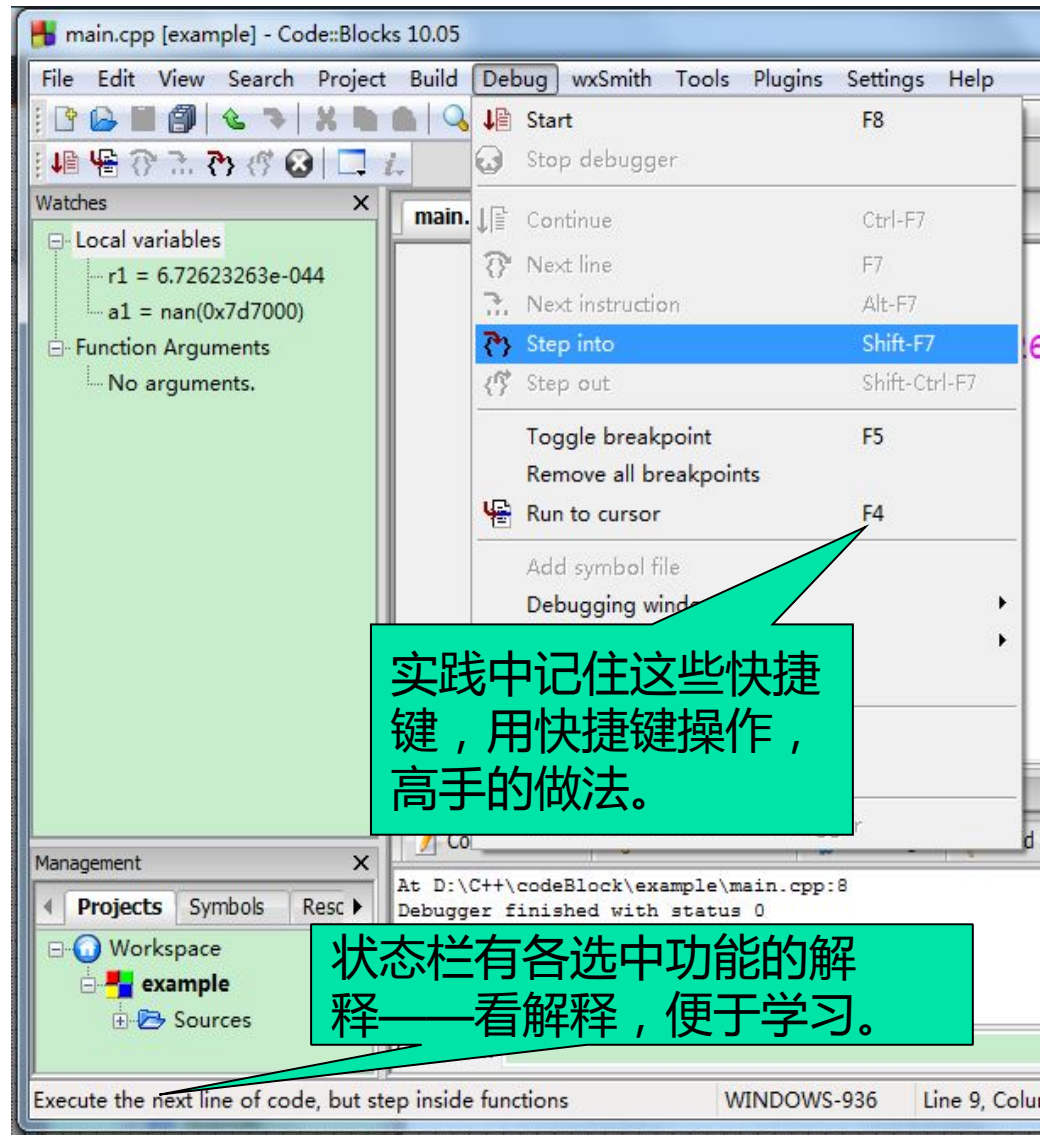
[illegible]

- ☞ 对于初学者，掌握所用的集成开发环境的一般用法，是一件非常重要的事情。
- ☞ 单步调试更是在实践中掌握的，做中学，学中记得用
- ☞ 本文讲Code::Blocks的单步调试，其他开发环境，也有类似功能，通一百通。
- ☞ 打开你的Code::Blocks，一边看，一边练.....



先认识菜单

- ❷ 单步调试功能在 Debug 菜单中
- ❷ 日常应用常用工具栏或快捷键，举步艰难时想起菜单，这里有全部的功能！
- ❷ 观察菜单，知道有哪些功能；
- ❷ 灰色的部分，会在“条件”具备时变得可用。

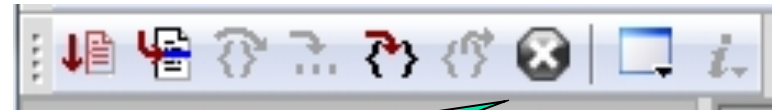
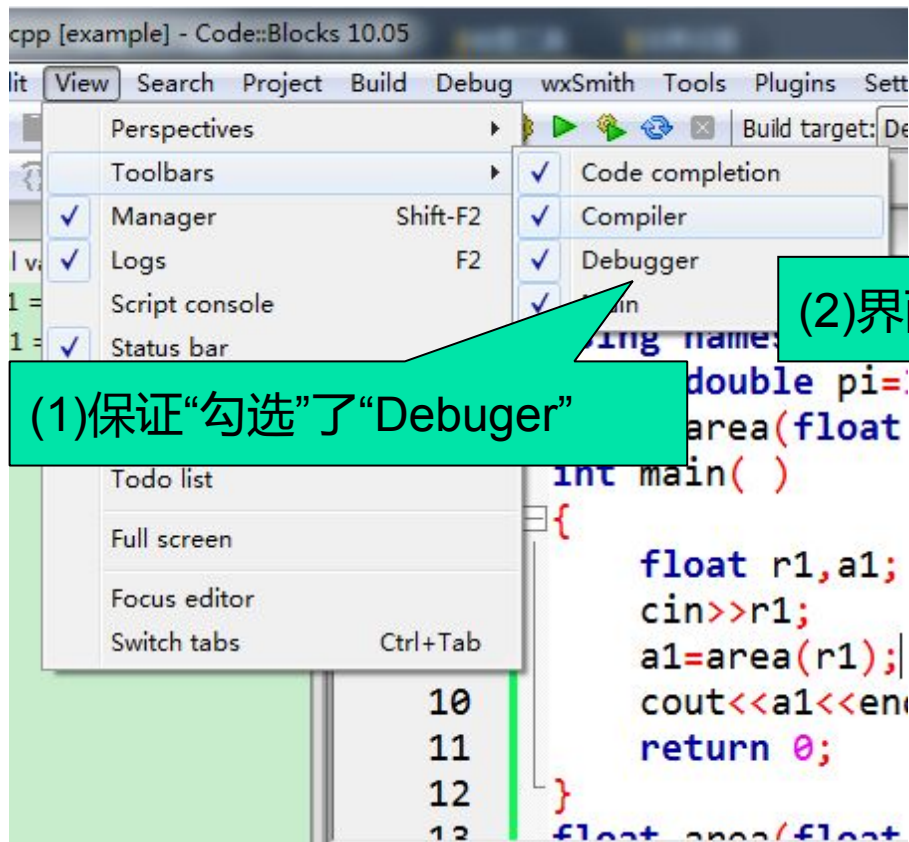




单步调试的工具栏

☞ 用工具栏中按钮，而不是菜单，更便捷

☞ 如何“找”出工具栏？



☞ 将鼠标“浮”在工具栏按钮上，会看到Run to Cursor、Next Line——菜单也有

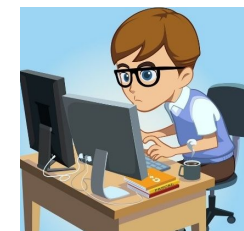
☞ 但，这儿更便捷！



技能1：用Run to Cursor、Next Line调试

[illegible]

🖱️ 记得要边看边练





(2)点Run to Cursor(或键盘F4)
(3)第7行前会有小黄三角，表现
执行到第7行

(1)光标先置到第7行

(5)程序执行到第8行，
小黄三角为证。

```
1 #include <iostream>
2 using namespace std;
3 const double pi=3.1415926;
4 int main( )
5 {
6     float r,a;
7     cout<<"输入半径: "<<endl;
8     cin>>r;
9     a = pi*r*r;
10    cout<<"输出面积: ";
11    cout<<a<<endl;
12    return 0;
13 }
```




(1)执行第8行时需要在此输入

(5)如果目的已经达到，那就
Stop Debugger。

输入半径:
2.5
输出面积:

(2) Watch窗口可以看到变量随程序执行的变化——
这是最珍贵的线索，无论找错，还是学习

(3)细心观察，你该有的品质。

(4)可以继续Next Line.....

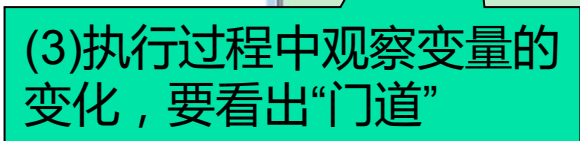
```
main.cpp x
1      #include <iostream>
2      using namespace std;
3      const double pi=3.1415926;
4      int main( )
5      {
6          float r,a;
7          cout<<"输入半径: "<<endl;
8          cin>>r;
9          a = pi*r*r;
10         cout<<"输出面积: ";
11         cout<<a<<endl;
```

Logs & others

Search results

(4)可以继续

```
At D:\C++\codeBlock\example\main.cpp:7
At D:\C++\codeBlock\example\main.cpp:8
At D:\C++\codeBlock\example\main.cpp:9
At D:\C++\codeBlock\example\main.cpp:10
At D:\C++\codeBlock\example\main.cpp:11
```





- (1)先执行几次程序，给出不同输入，发现总是2。
- (2)编译时会有一个warning，先不理睬。

(3)如果你已经看出了这个简单程序中的问题，也尝试一下，用单步调试发现问题。

(4)单步调试中，发现无论输入是多少，总会执行到这个分支。

(5)这个分支永远没有机会执行！

[illegible]

(2)却执行到了这个分支。



要跟踪这个程序的执行

```
#include <iostream>
using namespace std;
const double pi=3.1415926;
float area(float r);
int main( )
{
    float r1,a1;
    cin>>r1;
    a1=area(r1);
    cout<<a1<<endl;
    return 0;
}
float area(float r)
{
    float a;
    a = pi*r*r;
    return a;
}
```

(2) 用前面的办法跟踪时，此行执行Next Line，直接跳到下一行，将函数调用当作一个整体看待

(1) 程序中有了自定义函数

(3) 错误可能发生在自定义函数中，能不能“尾随”“跟踪”进函数内部来？

(4) 期待能够到函数里面看一看。



Step Into到函数中去

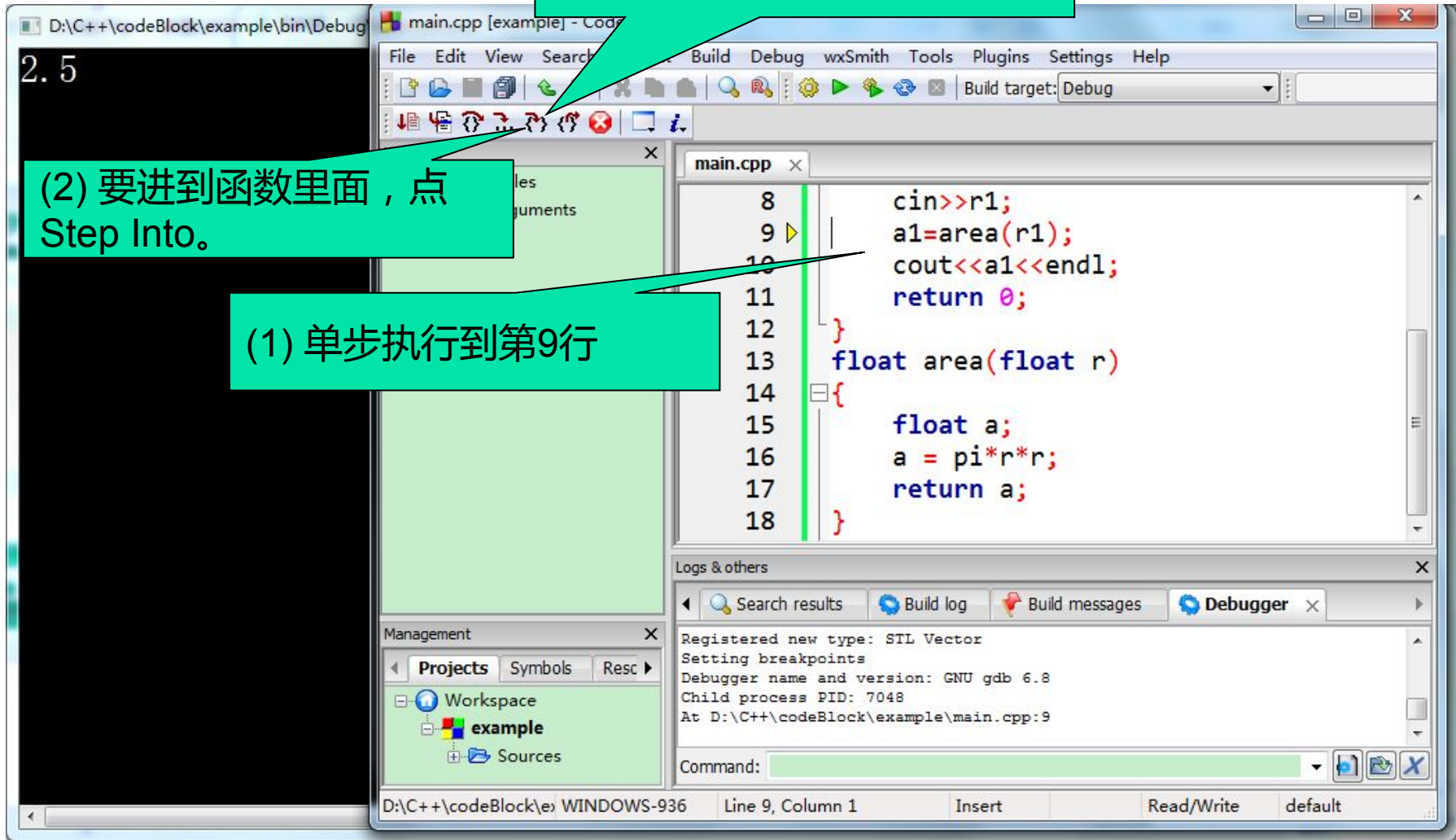
10101010000100101010101010101010101000110011001101010100001001

100101000011001100110101010000100101010

(3) 观察这些图标，很形象的。

(2) 要进到函数里面，点 Step Into。

(1) 单步执行到第9行





现在Into进来了

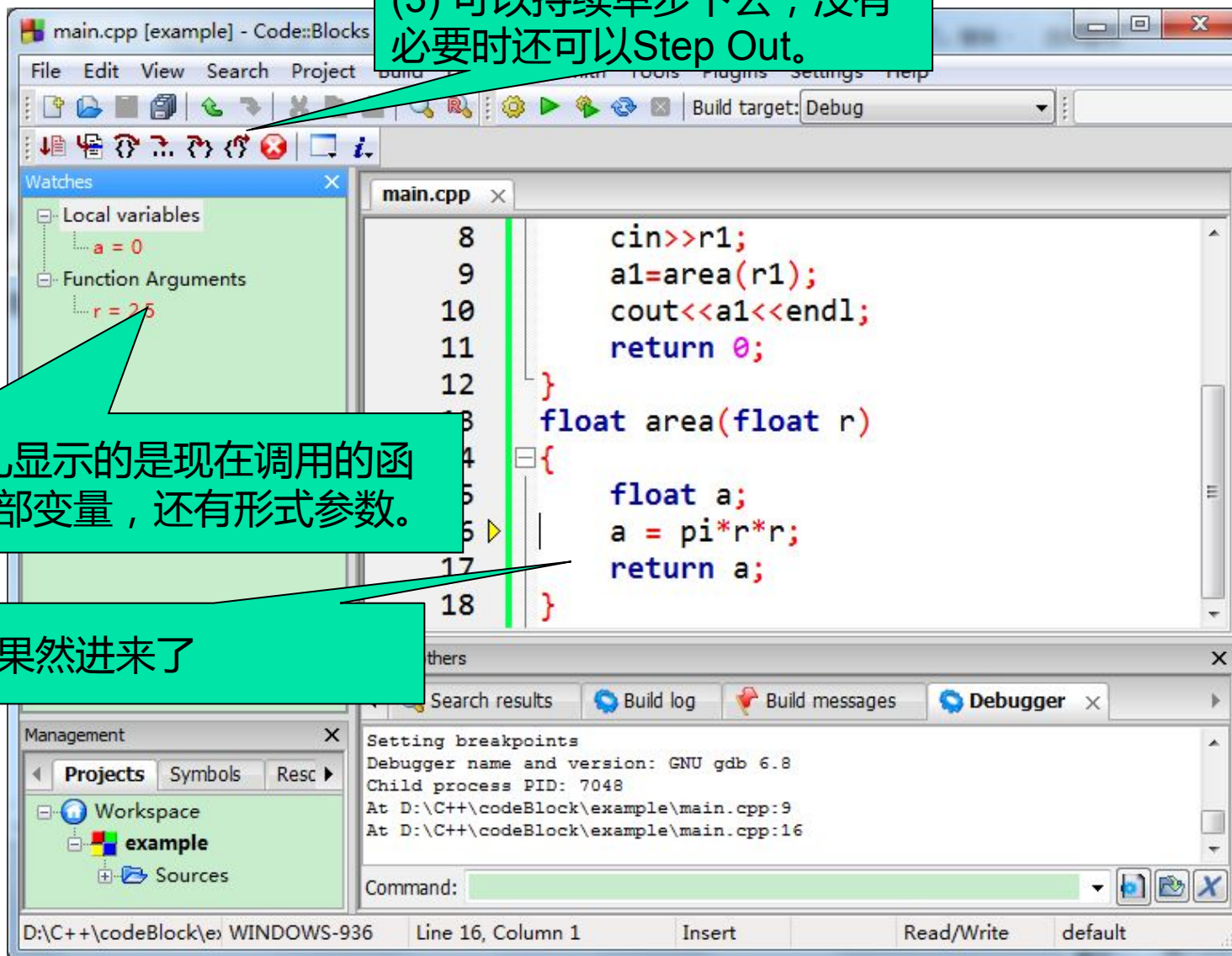
101010100001001010101010101010101010100001100110011010101010

(3) 可以持续单步下去，没有必要时还可以Step Out。

01010101001010000110011001101010101000010010101010

(2) 这儿显示的是现在调用的函数的局部变量，还有形式参数。

(1) 果然进来了





实践，检验技能的唯一标准

```
#include <iostream>
using namespace std;
float max(float x, float y);
int main ()
{
    float a,b,c;
    cin>>a>>b;
    c=max(a, b) ;
    cout<<"The max is "<<c<<endl;
    return 0;
}
float max(float x, float y)
{
    float z;
    z=(x<y)? x : y ;
    return  z;
}
```

☞ **这个程序本来要输出大值，
却总输出小值！**

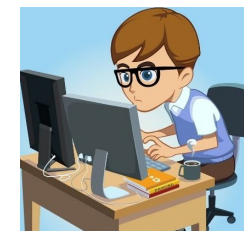
☞ 可能问题你已经看出来了，
跟踪进函数里面看一看吧。



技能3：会用断点

[illegible]

🖱️ 能“一步到位”的办法





断点(breakpoint)

🖱️ 单步执行

- 💾 每执行一步断一下，看到每一步到底干了些什么
- 💾 方式：Next Line/Step Into/Step Out等

🖱️ 运行到光标

- 💾 运行到光标所在位置，很方便
- 💾 光标所在位置，实际就是一个“断点”
- 💾 方式：Run to Cursor

🖱️ 设置断点

- 💾 任意指定到哪儿断开，想在哪儿停下来，就在哪儿停
- 💾 相比：单步太烦，运行到光标只能指定一个地方



要跟踪的程序

10101010000100101010101010101010101001010000110011001101010100001001010101010101010101001010000110011001101010100001001010101010101010101001010000110011001101010100001001010101010101010101001010000100101010

```
#include<iostream>
#include<cmath>
using namespace std;
int max(int,int);
int main( )
{
    int m,a,b;
    a=100;
    b=200;
    m=max(a,b);
    cout<<"最大:"<<m<<endl;
    return 0;
}
```

```
int max(int x,int y)
{
    int z;
    if(x>y)
        z=x;
    else
        z=y;
    return z;
}
```



(3) 断点设好后，点此“Debug/Continue”按钮

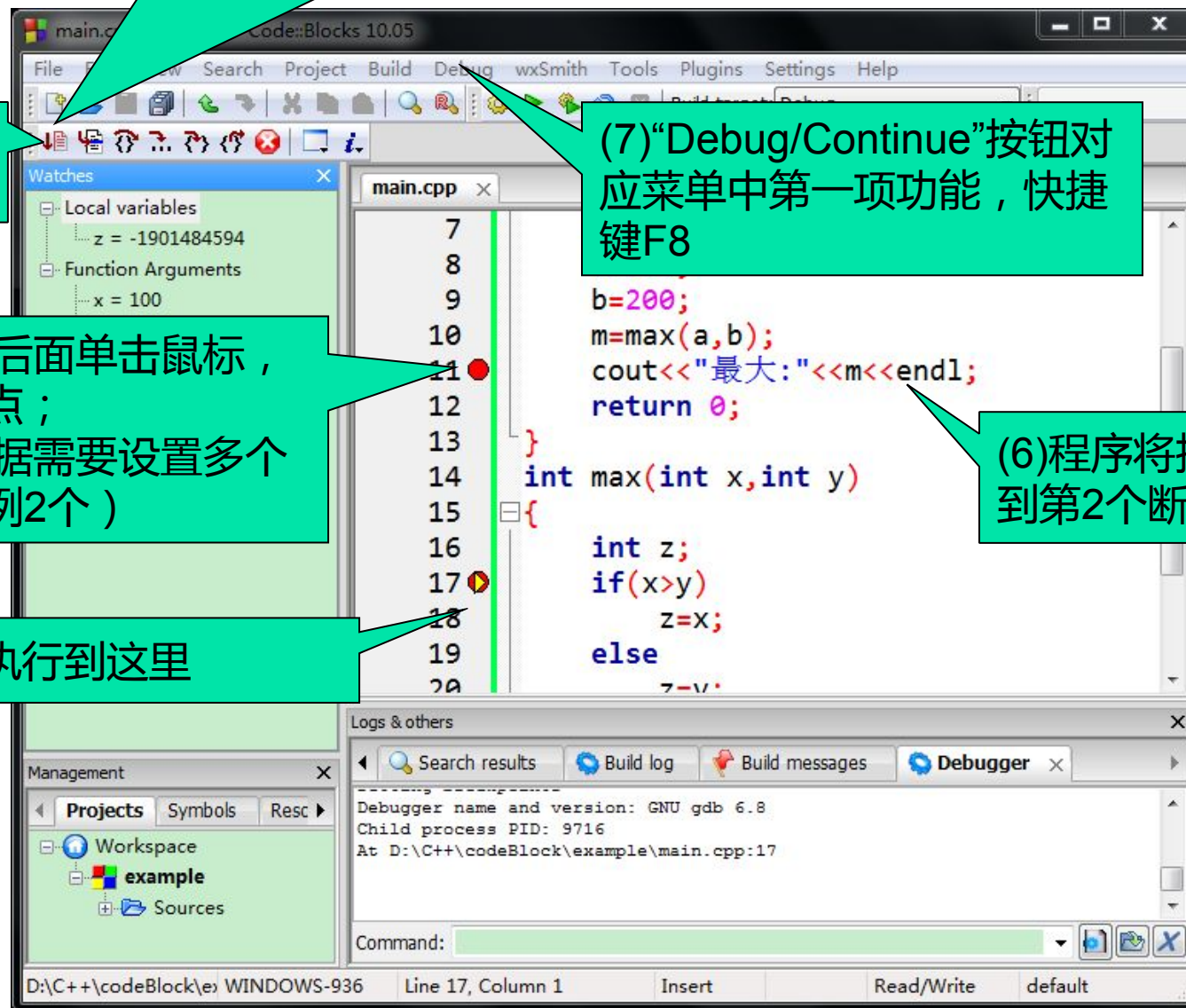
(5)再点这里

(1)在行号后面单击鼠标，
将设置断点；
(2)可以根据需要设置多个
断点（本例2个）

(4)程序执行到这里

(7)“Debug/Continue”按钮对
应菜单中第一项功能，快捷
键F8

(6)程序将执行
到第2个断点





```
#include <iostream>
using namespace std;
float max(float x, float y);
int main ()
{
    float a,b,c;
    cin>>a>>b;
    c=max(a, b) ;
    cout<<"The max is "<<c<<endl;
    return 0;
}

float max(float x, float y)
{
    float z;
    z=(x<y)? x : y ;
    return  z;
}
```

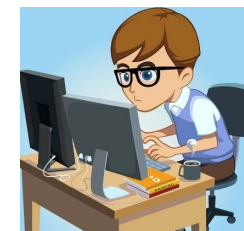
🖱️ **这个程序本来要输出大值，
却总输出小值！**

🖱️ 怀疑函数有问题，设置断点，直接进函数内部看一看吧。



结束语

10101010000100101010101010101010101001010000110011001101010100001001010101010101010101001010000110011001101010100001001010101010101010101001010000100101010101010101010010100001100110





- 借此简易讲义入手，
- 其他功能，自主探究，自然会用